

Bloom App — Database Relationships

A visual walkthrough of the Prisma schema in `backend/prisma8/contract.prisma`.
Generated 2026-10-09.

The schema has six models centered on **User**. Everything a user creates — habits, their daily entries, routines, and the steps inside routines — hangs off that root and is removed with the user via cascading deletes. The diagram below shows every table and the foreign keys that connect them.

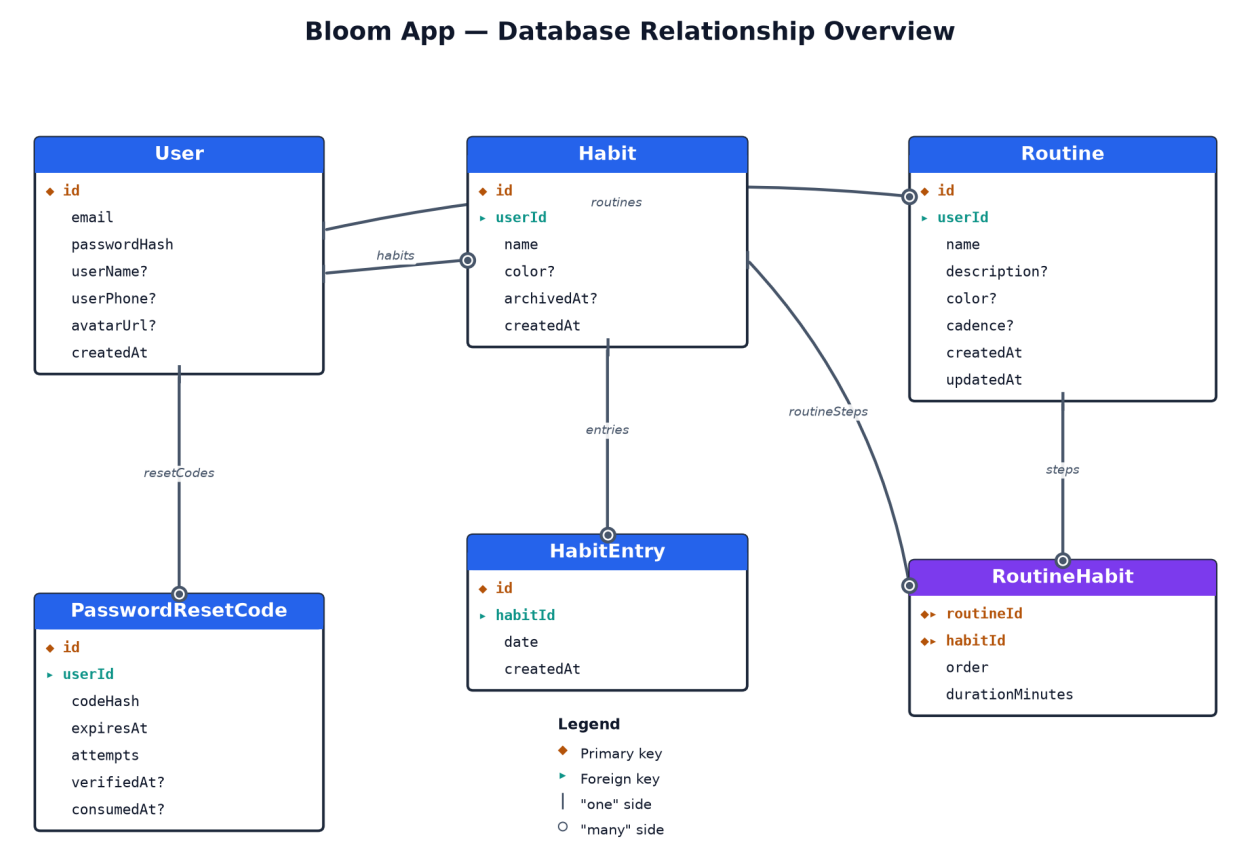


Figure 1 — Full entity-relationship overview. ♦ = primary key, ▴ = foreign key, | = “one” side, ○ = “many” side.

Relationships at a glance

Relationship	Cardinality	Key / On-delete
User → Habit	1 : many	Habit.userId → Cascade
User → Routine	1 : many	Routine.userId → Cascade
User → PasswordResetCode	1 : many	PasswordResetCode.userId → Cascade
Habit → HabitEntry	1 : many	HabitEntry.habitId → Cascade
Routine ↔ Habit	many : many	via RoutineHabit (join)
RoutineHabit → RoutineStepGuide	1 : many	composite FK → Cascade

The relationships in detail

1. User → Habit (one-to-many)

A **User** owns many **Habit** rows through the exposed `habits` `Habit[]` back-relation. The real database column lives on the child: `Habit.userId` references `User.id`. Deleting a user cascades down, removing all of their habits automatically.

```
user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

2. Habit → HabitEntry (one-to-many)

Each **Habit** has many **HabitEntry** rows — one per day the habit is logged. The unique composite index `@@index([habitId, date], unique: true)` enforces the core rule of the app: **a habit can be logged at most once per day**. Attempting a second entry for the same habit and date is rejected by the database.

3. User → Routine (one-to-many)

A **User** owns many **Routine** rows via `Routine.userId`, indexed by `routines_userId_idx` for fast “list my routines” queries. Note `updatedAt` has no default — the comment in the schema states `RoutineService` stamps it on every write.

4. User → PasswordResetCode (one-to-many)

A **User** may have several password-reset codes over time. Each code tracks its own attempts, `expiresAt`, and the `verifiedAt` / `consumedAt` lifecycle timestamps. Indexed by `(userId, createdAt)` to fetch the latest code for a user quickly.

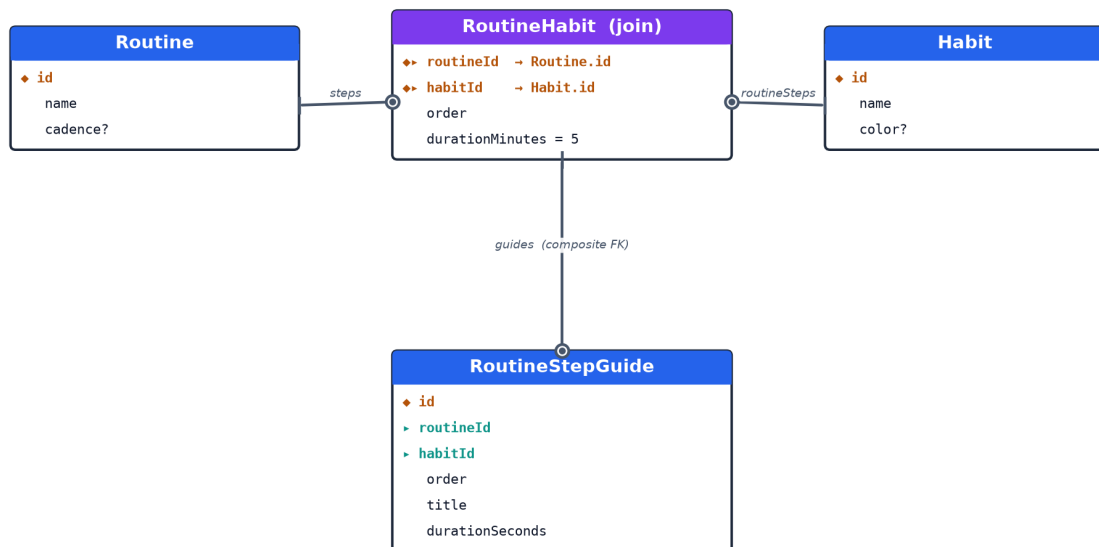
5. Routine ↔ Habit (many-to-many via RoutineHabit)

This is the heart of the schema. A routine is an ordered list of habits, and a habit can appear in many routines — a classic many-to-many. Instead of an implicit join, the schema models an **explicit join table**, `RoutineHabit`, because the link itself carries data:

- **order** — the habit’s position within the routine (step 1, 2, 3...)
- **durationMinutes** — how long that step should take (default 5)

Its primary key is the **composite** `@@id([routineId, habitId])`, so a given habit can appear in a routine only once, and that pair uniquely identifies the step. Both foreign keys cascade on delete.

Many-to-Many: Routine ↔ Habit + nested step guides



RoutineHabit uses a composite primary key (routineId, habitId).
RoutineStepGuide references BOTH columns to point at one step.

Figure 2 — The many-to-many join and the nested step guides below it.

6. RoutineHabit → RoutineStepGuide (one-to-many)

A single routine step can be broken into ordered sub-moves — the schema comment gives “Cat-Cow spinal rolls” as an example inside a stretching step. Because the parent’s primary key is the pair (`routineId`, `habitId`), each **RoutineStepGuide** must store **both** columns to point back at its step — a foreign key over two fields:

```
step RoutineHabit @relation( fields: [routineId, habitId], references: [routineId, habitId], onDelete: Cascade)
```

Each guide has its own `order`, `title`, and `durationSeconds` — note the step level is measured in *minutes* while guides are in *seconds*.

Cross-cutting design notes

- **Everything cascades on delete.** Removing a User tears down the whole tree: habits → entries, routines → steps → guides, and all reset codes. No orphan rows are left behind.
- **Client-generated IDs.** All `@default(uuid())` ids are generated by the client, not the database (per the header comment), so inserts don’t need a round-trip to learn their own id.
- **Selective indexing.** Many foreign keys carry `index: false` — Prisma is told not to auto-create an index because the column is already covered (e.g. by the composite `@@id` or a unique index) or the write cost isn’t worth it. Where lookups matter — `routine_habits_habitId_idx` — an explicit index is kept so “which routines use this habit?” stays fast.
- **Business rules enforced in the schema.** The unique (`habitId`, `date`) index and the composite `RoutineHabit` key push correctness into the database rather than relying on application code.